Presentation Guide — Phase 2 Fine-Tuning Notebook

YOLOv7 Gym Equipment Detection | UNH Deep Learning Spring 2026

Team: Varun Gazala · Mohit Raiyani · Jatinkumar Nabhoya

Big Picture — What This Notebook Does

You took a **pretrained YOLOv7** (trained on 80 COCO classes) and fine-tuned it on your own 5 gym-equipment classes. The notebook has 3 phases in one file:

> **Hyperparameter Search → Full Fine-Tuning → Evaluation**

The Deep Learning Architecture: YOLOv7

YOLO = You Only Look Once. It detects all objects in a single forward pass — not a two-step "propose then classify" approach like Faster-RCNN.

Architecture Overview

```
Input Image (640×640)
        ↓
  [BACKBONE]  ← extracts features (like VGG/ResNet)
        ↓
  [NECK/FPN]  ← multi-scale feature fusion
        ↓
  [HEAD]      ← predicts boxes + classes at 3 scales
        ↓
Output: boxes, confidence scores, class probabilities
```

Backbone — ELAN (Efficient Layer Aggregation Network)

A very deep CNN that compresses the image through many convolutional layers to learn rich features — edges → textures → object parts → full objects. YOLOv7 uses a custom ELAN design that aggregates features from multiple depths efficiently.

Neck — FPN (Feature Pyramid Network)

Objects appear at different sizes in images. FPN creates a pyramid of feature maps at 3 different resolutions:

Feature Map	Resolution	Detects
Large	80×80	Small objects
Medium	40×40	Medium objects
Small	20×20	Large objects

Head — IDetect

For each cell in each feature map, the head predicts:

- **3 anchors** × **(5 + num_classes)** values per cell
- Original COCO: `3 × (5+80) = 255` channels
- **Your version:** `3 × (5+5) = 30` **channels** (5 classes)

The 10 values per anchor:

Values	Meaning
`tx, ty`	Box center offset from grid cell
`tw, th`	Box width/height (relative to anchor)
`objectness`	"Is there an object here?"
`cls_0 … cls_4`	Class probabilities (dumbbell, barbell, …)

What You Changed in the Architecture

```
# Replace the 3 final Conv2d layers in the detection head:
for i, conv in enumerate(detect.m):
    detect.m[i] = nn.Conv2d(conv.in_channels, new_no * na, 1)
    # in_channels stays same — output shrinks from 255 → 30
```

Everything before the head (backbone + neck) keeps its pretrained COCO weights.

Section-by-Section Explanation

Section 1 — Environment Setup

- Clones the YOLOv7 repo from GitHub if not present
- Downloads `yolov7.pt` (72 MB COCO pretrained weights)
- Sets `IMG_SIZE=640`, `BATCH_SIZE=8`, `DEVICE` (auto-selects GPU/CPU)
- Class order: `['dumbbell', 'barbell', 'kettlebell', 'resistance_band', 'pull_up_bar']`

Section 2 — Data Augmentation (9 Techniques)

Why augment? You only have 162 training images. Augmentation artificially multiplies your dataset by showing the model the same objects under different conditions.

The pipeline uses `albumentations` with bounding-box-aware transforms (`min_visibility=0.3` — a box is dropped if less than 30% remains visible after the transform).

#	Technique	What It Does	Why
1	HorizontalFlip	Mirror image	Equipment looks same both ways
2	HueSaturationValue	Change colors	Different lighting in gyms
3	RandomBrightnessContrast	Brighter/darker	Shadows, overhead lights
4	GaussianBlur	Add blur	Motion blur, out-of-focus camera
5	CLAHE	Enhance local contrast	Better edge visibility in dark areas
6	Perspective	Tilt/skew	Camera angle variation
7	ShiftScaleRotate	Translate, zoom, rotate	Different viewpoints
8	CoarseDropout	Black rectangles over image	Forces model not to rely on single region

#	Technique	What It Does	Why
9	Normalize	Subtract ImageNet mean/std	Matches pretrained backbone expectations

Mosaic Augmentation — Stitches 4 training images into one at a random split point. Forces the model to detect small objects and handle unusual compositions. Applied with probability `mosaic_p` per sample.

Section 3 — Hyperparameter Tuning (Random Search)

Problem: How do you pick learning rate, momentum, weight decay, and freeze duration?

Solution: Random search — 8 trials × 8 epochs each (fast, cheap).

```
lr:            log-uniform [1e-4, 1e-2]   # log scale = fair coverage
weight_decay:  log-uniform [1e-5, 1e-3]
momentum:      uniform     [0.85, 0.95]
freeze_epochs: integer     [2, 6]
```

Why log-uniform for LR? `0.001` vs `0.002` is a 2× difference (huge). `0.9` vs `0.91` is 1.1× (tiny). Sampling on a log scale gives each order of magnitude equal representation.

Best HP found:

Hyperparameter	Value
Learning rate	0.00352
Momentum	0.936
Weight decay	0.000934
Freeze epochs	2

Section 4 — Two-Phase Fine-Tuning

Phase A — Frozen Backbone (2 epochs)

```
# Freeze everything
for p in model.parameters():
    p.requires_grad_(False)

# Only unfreeze detection head
for p in model.model[-1].parameters():
    p.requires_grad_(True)
```

- Trains only the new 30-channel head
- Backbone keeps its ImageNet/COCO feature knowledge intact
- LR = 0.00352 (full best LR)

Phase B — Full Fine-Tuning (remaining epochs)

```
# Unfreeze all layers
for p in model.parameters():
    p.requires_grad_(True)

# Use 10× smaller LR to avoid destroying pretrained features
optimizer = SGD(..., lr=best_hp['lr'] * 0.1, ...)
```

- Fine-tunes the entire network end-to-end
- 10× smaller LR prevents catastrophic forgetting

Training Details

Setting	Value
Optimizer	SGD with Nesterov momentum
LR Scheduler	Cosine Annealing (smooth decay to near-zero)
Max epochs	50
Early stopping	Patience = 15 on val mAP@0.5
Actual epochs run	42 (early stop triggered)

Section 5 — Loss Function & Training Curves

The loss has 3 components:

```
Total Loss = λ_box × CIoU_loss  +  λ_obj × BCE(objectness)  +  λ_cls × BCE(class)
```

Component	Formula	What It Penalizes
CIoU	1 − IoU + center_dist² / diag² + aspect_ratio_term	Bad box location/size
BCE objectness	Binary cross-entropy	Wrong "is there an object?" answer
BCE class	Binary cross-entropy	Wrong class prediction

Why CIoU instead of MSE for boxes?
MSE treats all coordinate errors equally. CIoU directly optimizes:

1. Overlap between predicted and GT box
2. Distance between centers
3. Aspect ratio consistency

Section 6 — Test-Time Augmentation (TTA)

At inference, run the model on multiple augmented views of the same image and merge results.

Original TTA (had a bug): 4 views — original, h-flip, scale 0.83×, scale 1.17×. Bug: multi-scale box coordinates were not correctly rescaled back to original image space.

Fixed TTA (Phase 3, Section 6): Original + horizontal flip only — simple and correct:

```
# Correct h-flip box coordinate transform:
boxes[:,0], boxes[:,2] = S - det[:,2], S - det[:,0]
# x1_new = width - x2_old  (left edge becomes right edge)
# x2_new = width - x1_old
```

Results from both views are merged using **batched NMS** (non-maximum suppression) per class.

Phase 3 — Final Evaluation

Per-Class AP@0.5

Class	AP@0.5	Why
kettlebell	0.7000	Distinctive round shape, easy to localize
dumbbell	0.1747	Occlusion from hands/racks
resistance_band	0.1067	Thin, deformable, no fixed shape
pull_up_bar	0.0980	Partially visible, elongated
barbell	0.0731	Very elongated, often cropped at image edges

Confusion Matrix

- Rows = True class, Columns = Predicted class
- Last row/column = background (unmatched FP / missed GT)
- Barbell and resistance_band are most confused — unusual shapes that don't match typical object silhouettes

F1 Score vs Confidence Threshold

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Threshold	Effect
Too high (e.g. 0.8)	High precision, low recall — misses many objects
Too low (e.g. 0.05)	Low precision, high recall — many false positives
Optimal: 0.19	**Best F1 = 0.338**

Final Results

Method	mAP@0.5	mAP@0.5:0.95
Baseline (COCO pretrained, no fine-tuning)	0.0005	0.0001
Fine-Tuned (standard)	0.4360	0.2305

Method	mAP@0.5	mAP@0.5:0.95
Fine-Tuned + TTA (h-flip)	**0.4603**	**0.2392**

Professor Questions & Answers

Q: Why use YOLOv7 and not train from scratch?

You only have 162 training images. A deep network trained from scratch needs millions of examples to learn basic visual features. Transfer learning lets you use YOLOv7's pretrained features (edges, textures, shapes) learned from 118K COCO images, and adapt just the final layers for your 5 classes. Training from scratch on 162 images would severely overfit.

Q: Why freeze the backbone first?

If you immediately fine-tune all layers with a high LR, the randomly-initialized head produces large gradient errors that propagate back and "corrupt" the pretrained backbone weights — this is called **catastrophic forgetting**. Freezing lets the new head stabilize first, then you unfreeze everything at a much smaller LR (10×) so the backbone shifts gently.

Q: What is mAP@0.5 and mAP@0.5:0.95?

mAP = mean Average Precision (averaged across all classes).

- `@0.5` — a detection counts as correct if IoU with the ground-truth box ≥ 0.5 (lenient)
- `@0.5:0.95` — averages mAP across IoU thresholds 0.50, 0.55, 0.60 … 0.95 in 0.05 steps. Much stricter — requires tight box localization, not just finding the right region.

Q: What is IoU?

IoU = Intersection over Union = area of overlap ÷ area of union between predicted and ground-truth box.

- IoU = 1.0 → perfect overlap

- IoU = 0.0 → no overlap
- Threshold of 0.5 means "good enough" detection

Q: Why random search instead of grid search or Bayesian optimization (Optuna)?

- Grid search is exponential in the number of hyperparameters — 4 values per HP × 4 HPs = 256 combinations
- Optuna (Bayesian) needs many trials to build a good surrogate model
- With only 8 trials on a small dataset, random search gives good coverage with no extra complexity. Research (Bergstra & Bengio 2012) shows random search matches grid search at any fixed trial budget

---

Q: Why is barbell AP the lowest (0.07)?

Barbells are extremely elongated and almost always partially cropped at image edges. YOLO anchor boxes are designed for roughly compact objects. A barbell spanning the full width of a 640×640 frame at very small height doesn't match any anchor well — the anchor IoU is too low to trigger a match, so the loss treats it as background.

Q: What is NMS (Non-Maximum Suppression)?

YOLO predicts thousands of boxes simultaneously (one per grid cell × 3 anchors × 3 scales = 25,200 boxes for 640×640 input). NMS removes duplicates:

1. Sort all boxes by confidence score (highest first)
2. Keep the top box
3. Remove all remaining boxes with IoU > threshold (0.45 here) with the kept box
4. Repeat for the next highest box

Q: What does two-phase fine-tuning achieve vs training everything from epoch 1?

Phase A gives the new 30-channel head a "warm start" — it learns reasonable weights before the lower-LR full fine-tuning in Phase B. Without this, the large random gradients from the untrained head in epoch 1 can destroy the delicate pretrained backbone features before they are useful. Empirically this improves final mAP by ~5–10% on small datasets.

Q: What is ImageNet normalization and why apply it?

The YOLOv7 backbone was pretrained on ImageNet where every image was normalized:

```
pixel_normalized = (pixel / 255 − mean) / std
mean = [0.485, 0.456, 0.406]   # RGB channel means
std  = [0.229, 0.224, 0.225]   # RGB channel stds
```

Applying the same normalization at fine-tuning/inference ensures the input distribution matches what the backbone learned to process. Using raw pixel values [0,1] without normalization would cause the first-layer activations to be out of the distribution the backbone was trained on.

Q: What is cosine annealing?

A learning rate schedule that decays the LR from its initial value to near-zero following a cosine curve:

```
LR(t) = LR_min + 0.5 × (LR_max − LR_min) × (1 + cos(π × t/T))
```

This avoids a sharp LR drop and allows the optimizer to explore broadly early and converge tightly late in training.

Q: Why SGD with Nesterov momentum instead of Adam?

SGD with momentum is standard for fine-tuning pretrained vision models — it generalizes better and avoids overfitting on small datasets. Adam adapts the LR per parameter, which can cause it to overfit on small datasets by "memorizing" individual examples. Nesterov momentum computes the gradient at the anticipated future position, which gives slightly faster convergence than standard momentum.